

HuggingFace를 활용한

인공지능 웹 애플리케이션 개발

제 3장

자연어 처리 웹 서비스 만들기

이 장의 학습 목표

FastAPI와 Hugging Face NLP 모델을 연동하여 실제 동작하는 웹 API 서비스를 구현합니다.

챗봇, 요약기, 번역기, 문서 유사도 검색기 등 핵심 NLP 서비스를 직접 만들어봅니다.

Swagger UI를 통해 프론트엔드 없이 AI API를 체험하고 검증하는 방법을 익힙니다.

목 차

제3장 자연어 처리 웹 서비스 만들기

★ 이 장의 개요

이 장에서는 Hugging Face의 다양한 자연어처리(NLP) 모델을 FastAPI와 연동하여 실제 웹 API 서비스 형태로 구현합니다. 단순한 챗봇부터 시작해서 요약기, 번역기, 문서 유사도 검색기까지 실습하며, 사용자 요청을 받아 모델이 예측을 수행하고 결과를 응답하는 AI 웹 서비스의 전체 흐름을 이해하게 됩니다. 또한 Swagger UI 기반으로 프론트엔드 없이 API를 체험하고 REST API 설계와 데이터 흐름에 집중합니다.

자연어 처리(NLP, Natural Language Processing)는 인공지능 분야에서 가장 활발히 응용되고 있는 영역 중 하나입니다. 텍스트를 이해하고, 변환하고, 생성하는 NLP 기술은 챗봇, 번역, 검색, 요약 등 수많은 실용 서비스의 핵심 엔진으로 자리 잡았습니다.

Hugging Face의 Transformers 라이브러리는 BERT, GPT-2, T5, MarianMT, sentence-transformers 등 수백 가지 NLP 모델을 단일 API로 제공하며, FastAPI와 결합하면 이러한 모델들을 빠르게 웹 서비스로 배포할 수 있습니다.

이 장에서 구현할 서비스 목록은 다음과 같습니다.

구현 서비스	사용 모델 / 기술
3.1 챗봇 API	KoGPT / GPT-2, DialoGPT (텍스트 생성)
3.2 문서 요약 API	T5, KETI-AIR/ke-t5-base (Seq2Seq)
3.3 번역 API	Helsinki-NLP/opus-mt-en-ko (MarianMT)
3.4 유사도 검색 API	sentence-transformers + FAISS
3.5 Swagger UI 체험	FastAPI 내장 /docs, /redoc

■ 3장 프로젝트 디렉터리 구조

chapter3-nlp-services/

└── main.py # FastAPI 앱 진입점

```
|—— requirements.txt
|—— .env                # HF_TOKEN 등 환경변수
|—— routers/
|   |—— chatbot.py      # 3.1 챗봇 라우터
|   |—— summarizer.py  # 3.2 요약기 라우터
|   |—— translator.py  # 3.3 번역기 라우터
|   └—— similarity.py   # 3.4 유사도 검색 라우터
|—— services/
|   |—— chatbot_service.py
|   |—— summarizer_service.py
|   |—— translator_service.py
|   └—— similarity_service.py
|—— models/
|   └—— schemas.py      # Pydantic 공통 모델
└—— data/
    └—— sample_docs.json # 유사도 검색용 샘플 문서
```

3.1 FastAPI 챗봇 만들기 (단순 텍스트 입출력)

챗봇(Chatbot)은 사용자의 텍스트 입력을 받아 자연스러운 응답을 생성하는 가장 직관적인 NLP 서비스입니다. 이 절에서는 Hugging Face의 텍스트 생성 모델을 FastAPI로 래핑하여 REST API 형태의 챗봇을 만듭니다.

3.1.1 챗봇의 기본 원리: 입력 텍스트 → 모델 추론 → 응답 텍스트

챗봇의 핵심 동작 원리는 단순합니다. 사용자가 입력한 텍스트(prompt)를 언어 모델에게 전달하면, 모델은 사전 학습된 지식을 기반으로 다음에 이어질 확률이 높은 토큰(token)들을 순서대로 생성하여 응답 텍스트를 만들어냅니다.

단계	설명	코드 역할
① 사용자 입력	HTTP POST 요청으로 텍스트 전달	FastAPI 엔드포인트가 수신
② 토크나이징	텍스트 → 토큰 ID 시퀀스 변환	<code>AutoTokenizer.encode()</code>
③ 모델 추론	토큰 시퀀스 → 다음 토큰 예측	<code>model.generate()</code>
④ 디코딩	토큰 ID 시퀀스 → 텍스트 변환	<code>tokenizer.decode()</code>
⑤ 응답 반환	JSON 형태로 클라이언트에 전달	FastAPI Response

텍스트 생성 모델의 동작 방식을 더 깊이 이해하려면 '생성 전략(Decoding Strategy)'에 대해 알아야 합니다. 어떤 전략을 선택하느냐에 따라 생성되는 텍스트의 품질과 다양성이 크게 달라집니다.

생성 전략	특징 및 사용 상황
Greedy Search	매 스텝마다 확률이 가장 높은 토큰 선택. 빠르지만 반복적이고 단조로운 텍스트 생성.
Beam Search	여러 후보 시퀀스를 동시에 탐색. 품질 높은 번역, 요약에 적합. <code>num_beams</code> 파라미터로 조절.
Top-k Sampling	확률 상위 k개 토큰 중 무작위 선택. 다양하고 창의적인 텍스트. k 값이 수록 다양성 증가.
Top-p (Nucleus) Sampling	누적 확률 p% 이내 토큰 중 선택. Top-k보다 유연하고 자연스러운 텍스트 생성.

생성 전략	특징 및 사용 상황
	트 생성.
Temperature	로짓 분포를 스케일링. 낮을수록(0~1) 집중, 높을수록(>1) 다양하고 창의적.

3.1.2 KoGPT 또는 GPT-2 기반 챗봇 구현

실습에서는 두 가지 모델을 사용합니다. 영어 챗봇에는 Microsoft의 DialoGPT를, 한국어 챗봇에는 SKT-AI의 KoGPT2를 활용합니다. 모델에 따라 입력 포맷과 생성 파라미터를 조금씩 다르게 설정해야 합니다.

📁 사용 모델 정보

microsoft/DialoGPT-medium

- 영어 대화에 특화된 GPT-2 기반 모델
- 아키텍처: GPT-2 (124M ~ 345M 파라미터)
- 학습 데이터: Reddit 대화 데이터 147M 쌍
- 라이선스: MIT

skt/kogpt2-base-v2

- 한국어 GPT-2 기반 생성 모델
- 아키텍처: GPT-2 (124M 파라미터)
- 학습 데이터: 모두의말뭉치, CC-100-Kor, 나무위키 등 40GB
- 라이선스: Apache 2.0

먼저 챗봇 서비스 로직을 작성합니다. 서비스 레이어에서 모델을 로드하고 텍스트를 생성하는 핵심 로직을 구현합니다.

services/chatbot_service.py

```
# services/chatbot_service.py
import torch
from transformers import (AutoTokenizer, AutoModelForCausalLM,
```

```
AutoModelForSeq2SeqLM)
from typing import Optional

class ChatbotService:
    """텍스트 생성 기반 챗봇 서비스"""

    def __init__(self):
        self.en_tokenizer = None
        self.en_model = None
        self.ko_tokenizer = None
        self.ko_model = None
        self.device = 'cuda' if torch.cuda.is_available() else 'cpu'

    def load_english_model(self):
        """DialogPT-medium 로드 (영어 챗봇)"""
        model_id = 'microsoft/DialogPT-medium'
        print(f'[INFO] Loading English chatbot: {model_id}')
        self.en_tokenizer = AutoTokenizer.from_pretrained(model_id)
        self.en_model = AutoModelForCausalLM.from_pretrained(
            model_id,
            torch_dtype=torch.float16 if self.device == 'cuda' else torch.float32
        ).to(self.device)
        self.en_model.eval()
        print('[INFO] English chatbot loaded.')

    def load_korean_model(self):
        """KoGPT2 로드 (한국어 챗봇)"""
        model_id = 'skt/kogpt2-base-v2'
        print(f'[INFO] Loading Korean chatbot: {model_id}')
        self.ko_tokenizer = AutoTokenizer.from_pretrained(
            model_id,
            bos_token='</s>', eos_token='</s>', unk_token='<unk>',
            pad_token='<pad>', mask_token='<mask>'
        )
        self.ko_model = AutoModelForCausalLM.from_pretrained(
            model_id
        ).to(self.device)
```

```
self.ko_model.eval()

print('[INFO] Korean chatbot loaded.')

def generate_english_response(
    self,
    user_input: str,
    max_length: int = 200,
    temperature: float = 0.7,
    top_p: float = 0.9,
) -> str:
    """DialogPT를 이용한 영어 응답 생성"""
    if self.en_model is None:
        raise RuntimeError('English model not loaded.')

    # DialogPT 형식: 입력 + EOS 토큰 추가
    input_ids = self.en_tokenizer.encode(
        user_input + self.en_tokenizer.eos_token,
        return_tensors='pt'
    ).to(self.device)

    with torch.no_grad():
        output_ids = self.en_model.generate(
            input_ids,
            max_length=max_length,
            pad_token_id=self.en_tokenizer.eos_token_id,
            do_sample=True,
            temperature=temperature,
            top_p=top_p,
            repetition_penalty=1.3, # 반복 억제
        )

    # 입력 부분을 제외한 응답만 디코딩
    response_ids = output_ids[:, input_ids.shape[-1]:]
    response = self.en_tokenizer.decode(
        response_ids[0],
        skip_special_tokens=True
    )
```

```
return response.strip()

def generate_korean_response(
    self,
    user_input: str,
    max_length: int = 128,
    temperature: float = 0.8,
    top_k: int = 50,
) -> str:
    """KoGPT2를 이용한 한국어 응답 생성"""
    if self.ko_model is None:
        raise RuntimeError('Korean model not loaded.')

    # KoGPT2 형식: <usr> 입력 <sys> 패턴
    prompt = f'<usr> {user_input} <sys> '
    input_ids = self.ko_tokenizer.encode(
        prompt, return_tensors='pt'
    ).to(self.device)

    with torch.no_grad():
        output_ids = self.ko_model.generate(
            input_ids,
            max_length=max_length,
            do_sample=True,
            temperature=temperature,
            top_k=top_k,
            repetition_penalty=2.0,
            pad_token_id=self.ko_tokenizer.pad_token_id,
            eos_token_id=self.ko_tokenizer.eos_token_id,
        )

    generated = self.ko_tokenizer.decode(
        output_ids[0],
        skip_special_tokens=True
    )
    # <sys> 이후 텍스트만 추출
    if '<sys>' in generated:
```

```

        response = generated.split('<sys>')[-1].strip()
    else:
        response = generated[len(prompt):].strip()
    return response

# 싱글턴 인스턴스
chatbot_service = ChatbotService()

```

3.1.3 대화 이력을 저장하지 않는 단순 채팅 구조

이 절에서 구현하는 챗봇은 '무상태(Stateless)' 구조입니다. 즉, 각 API 요청은 이전 대화 이력을 기억하지 않고 독립적으로 처리됩니다. 이는 서버 구현을 단순하게 유지하고, 수평 확장(Horizontal Scaling)이 용이하다는 장점이 있습니다.

구조 유형	특징
무상태 챗봇 (이 절 구현)	각 요청이 독립적. 이전 대화 기억 없음. 서버 상태 관리 불필요. 확장성이.
상태 유지 챗봇 (심화)	대화 히스토리를 서버(Redis 등) 또는 클라이언트에 저장. 문맥 연속성 원.

💡 다음 챕터에서 다룰 내용

대화 이력을 유지하는 멀티턴(Multi-turn) 챗봇은 이 교재의 후속 챕터에서 다룹니다. session_id 기반의 대화 상태 관리와 Redis를 이용한 히스토리 저장 방법을 배울 예정입니다.

3.1.4 POST /chat API로 사용자 입력 처리

이제 FastAPI 라우터를 작성합니다. 클라이언트는 POST /chat 엔드포인트로 사용자 메시지와 언어 설정을 전송하고, 서버는 해당 언어의 생성 모델을 통해 응답을 반환합니다.

models/schemas.py - Pydantic 공통 스키마

```

# models/schemas.py
from pydantic import BaseModel, Field
from typing import Optional, List

```

```
from enum import Enum

class Language(str, Enum):
    korean = 'ko'
    english = 'en'

# ——— 챗봇 스키마 —————
class ChatRequest(BaseModel):
    message: str = Field(
        ...,
        min_length=1,
        max_length=500,
        description='사용자 입력 메시지 (최대 500자)',
    )
    language: Language = Field(
        Language.english,
        description='응답 언어: ko(한국어) / en(영어)',
    )
    max_length: int = Field(200, ge=50, le=500,
        description='생성 최대 토큰 길이')
    temperature: float = Field(0.7, ge=0.1, le=1.5,
        description='생성 다양성 조절 (낮을수록 결정론적)')

class Config:
    json_schema_extra = {
        'example': {
            'message': 'What is artificial intelligence?',
            'language': 'en',
            'max_length': 200,
            'temperature': 0.7
        }
    }

class ChatResponse(BaseModel):
    user_message: str
    bot_response: str
    language: str
```

```
model_used: str
tokens_generated: Optional[int] = None
```

routers/chatbot.py

```
# routers/chatbot.py
from fastapi import APIRouter, HTTPException, status
from models.schemas import ChatRequest, ChatResponse, Language
from services.chatbot_service import chatbot_service
import time

router = APIRouter(prefix='/chat', tags=['Chatbot'])

@router.post(
    '/',
    response_model=ChatResponse,
    summary='챗봇 응답 생성',
    description='사용자 메시지를 입력하면 AI 챗봇이 응답을 생성합니다.',
)
async def chat(
    request: ChatRequest,
):
    start_time = time.time()

    try:
        if request.language == Language.korean:
            if chatbot_service.ko_model is None:
                raise HTTPException(
                    status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
                    detail='한국어 챗봇 모델이 로드되지 않았습니다.',
                )
            response = chatbot_service.generate_korean_response(
                user_input=request.message,
                max_length=request.max_length,
                temperature=request.temperature,
            )
            model_used = 'skt/kogpt2-base-v2'
```

```
else: # English
    if chatbot_service.en_model is None:
        raise HTTPException(
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
            detail='영어 챗봇 모델이 로드되지 않았습니다.',
        )
    response = chatbot_service.generate_english_response(
        user_input=request.message,
        max_length=request.max_length,
        temperature=request.temperature,
    )
    model_used = 'microsoft/DialoGPT-medium'

    if not response:
        response = '죄송합니다. 응답을 생성하지 못했습니다.'

    return ChatResponse(
        user_message=request.message,
        bot_response=response,
        language=request.language.value,
        model_used=model_used,
        tokens_generated=len(response.split()),
    )

except HTTPException:
    raise
except Exception as e:
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f'챗봇 처리 중 오류 발생: {str(e)}',
    )

@router.get('/health', summary='챗봇 모델 상태 확인')
async def chatbot_health():
    return {
        'en_model_loaded': chatbot_service.en_model is not None,
```

```
'ko_model_loaded': chatbot_service.ko_model is not None,
'device': chatbot_service.device,
}
```

main.py - 앱 진입점과 startup 이벤트

```
# main.py
from contextlib import asynccontextmanager
from fastapi import FastAPI
from routers import chatbot, summarizer, translator, similarity
from services.chatbot_service import chatbot_service
from services.summarizer_service import summarizer_service
from services.translator_service import translator_service
from services.similarity_service import similarity_service

@asynccontextmanager
async def lifespan(app: FastAPI):
    # — 서버 시작 시 모델 일괄 로드 —
    print('[STARTUP] Loading all NLP models...')
    chatbot_service.load_english_model()
    chatbot_service.load_korean_model()
    summarizer_service.load_model()
    translator_service.load_models()
    similarity_service.load_model()
    similarity_service.build_index() # FAISS 인덱스 구축
    print('[STARTUP] All models loaded!')
    yield
    # — 서버 종료 시 정리 —
    print('[SHUTDOWN] Cleaning up...')

app = FastAPI(
    title='NLP Web Services API',
    description='Hugging Face NLP 모델 기반 웹 서비스 모음',
    version='1.0.0',
    lifespan=lifespan,
)
```

```
# 라우터 등록
app.include_router(chatbot.router)
app.include_router(summarizer.router)
app.include_router(translator.router)
app.include_router(similarity.router)

@app.get('/', tags=['Health'])
async def root():
    return {'status': 'NLP Services Running', 'docs': '/docs'}
```

테스트 실행 결과를 확인해 봅니다.

```
# 서버 실행
$ uvicorn main:app --reload

# curl 테스트 (영어 챗봇)
$ curl -X POST http://localhost:8000/chat/ ₩
-H 'Content-Type: application/json' ₩
-d '{"message": "Tell me about AI", "language": "en"}'
```

```
# 응답 예시:
{
  "user_message": "Tell me about AI",
  "bot_response": "AI is a fascinating field that involves ...",
  "language": "en",
  "model_used": "microsoft/DialoGPT-medium",
  "tokens_generated": 18
}
```

```
# curl 테스트 (한국어 챗봇)
$ curl -X POST http://localhost:8000/chat/ ₩
-H 'Content-Type: application/json' ₩
-d '{"message": "인공지능이란 무엇인가요?", "language": "ko"}'
```

⚠ 모델 메모리 최적화 팁

GPU 메모리가 부족한 환경에서는 모든 모델을 동시에 로드하면 OOM(Out of Memory) 오류가

발생할 수 있습니다.

아래 방법으로 메모리 사용량을 줄이세요.

1. 8비트 양자화 (bitsandbytes 필요):

```
model = AutoModelForCausalLM.from_pretrained(model_id, load_in_8bit=True)
```

2. CPU 오프로딩 (device_map 활용):

```
model = AutoModelForCausalLM.from_pretrained(model_id, device_map='auto')
```

3. 필요한 모델만 지연 로드 (Lazy Loading):

startup 이벤트 대신 첫 요청 시점에 모델 로드

3.2 요약기 API 만들기 (T5 기반)

문서 요약(Text Summarization)은 긴 텍스트에서 핵심 내용을 추출하거나 재구성하여 짧은 형태로 만드는 NLP 태스크입니다. 이 절에서는 T5(Text-to-Text Transfer Transformer) 아키텍처를 기반으로 한 요약 모델을 FastAPI로 구현합니다.

3.2.1 T5 모델 개념 소개 ('summarize:' prefix 사용)

T5는 2019년 Google이 발표한 모델로, '모든 NLP 태스크를 텍스트→텍스트 변환 문제로 통일'한다는 혁신적인 아이디어를 제시했습니다. 번역, 요약, 분류, 질의응답 등 다양한 태스크를 별도의 출력 헤드 없이 단일 Seq2Seq 구조로 처리합니다.

T5의 핵심은 태스크 prefix입니다. 입력 텍스트 앞에 어떤 태스크를 수행할지를 나타내는 텍스트 접두사를 붙여 모델에게 의도를 전달합니다.

T5 태스크 Prefix	예시 입력 형식
요약 (Summarize)	summarize: The stock market fell sharply today...
번역 (Translate)	translate English to German: Hello, how are you?
문법 교정 (Grammar)	grammar: She go to school yesterday.
질의응답 (Q & A)	question: What is the capital? context: Paris is...

T5 주요 변형 모델

t5-small : 60M 파라미터. 가볍고 빠름. 학습/테스트 환경에 적합.
 t5-base : 220M 파라미터. 성능과 속도의 균형.
 t5-large : 770M 파라미터. 높은 품질. GPU 권장.
 google/flan-t5-base : instruction-tuned 버전. 더 다양한 태스크 지원.
 KETI-AIR/ke-t5-base : 한국어 특화 T5. 한글 텍스트 요약에 최적화.

3.2.2 한글 뉴스 요약 모델 활용 실습 (KETI-AIR/ke-t5-base)

한국어 요약을 위해 KETI-AIR(한국전자기술연구원)에서 공개한 ke-t5-base 모델을 활용합니다. 이 모델은 T5 아키텍처에 한국어 데이터로 사전 학습되어, 한글 뉴스, 보고서 등의 요약에 효과적입니다.

services/summarizer_service.py

```
# services/summarizer_service.py
import torch
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
from typing import Optional

class SummarizerService:
    """T5 기반 문서 요약 서비스"""

    # 영어 요약: facebook/bart-large-cnn 또는 t5-base
    EN_MODEL_ID = 'facebook/bart-large-cnn'
    # 한국어 요약: KETI-AIR/ke-t5-base
    KO_MODEL_ID = 'KETI-AIR/ke-t5-base'

    def __init__(self):
        self.en_tokenizer = None
        self.en_model = None
        self.ko_tokenizer = None
        self.ko_model = None
        self.device = 'cuda' if torch.cuda.is_available() else 'cpu'

    def load_model(self):
        """영어/한국어 요약 모델 로드"""
        print(f'[INFO] Loading English summarizer: {self.EN_MODEL_ID}')
        self.en_tokenizer = AutoTokenizer.from_pretrained(self.EN_MODEL_ID)
        self.en_model = AutoModelForSeq2SeqLM.from_pretrained(
            self.EN_MODEL_ID
        ).to(self.device)
        self.en_model.eval()

        print(f'[INFO] Loading Korean summarizer: {self.KO_MODEL_ID}')
```

```
self.ko_tokenizer = AutoTokenizer.from_pretrained(self.KO_MODEL_ID)
self.ko_model = AutoModelForSeq2SeqLM.from_pretrained(
    self.KO_MODEL_ID
).to(self.device)
self.ko_model.eval()
print('[INFO] Both summarizers loaded.')
```

```
def summarize_english(
    self,
    text: str,
    max_length: int = 150,
    min_length: int = 40,
    num_beams: int = 4,
) -> dict:
```

```
    """BART를 이용한 영어 문서 요약"""
```

```
    inputs = self.en_tokenizer(
        text,
        return_tensors='pt',
        max_length=1024,
        truncation=True,
    ).to(self.device)
```

```
    with torch.no_grad():
        summary_ids = self.en_model.generate(
            inputs['input_ids'],
            attention_mask=inputs['attention_mask'],
            max_length=max_length,
            min_length=min_length,
            num_beams=num_beams,
            length_penalty=2.0, # 긴 요약 선호
            early_stopping=True,
            no_repeat_ngram_size=3,
        )
```

```
    summary = self.en_tokenizer.decode(
        summary_ids[0],
        skip_special_tokens=True,
```

```

    )
    input_tokens = inputs['input_ids'].shape[1]
    output_tokens = summary_ids.shape[1]
    return {
        'summary': summary,
        'input_tokens': input_tokens,
        'output_tokens': output_tokens,
        'compression_ratio': round(input_tokens / max(output_tokens, 1), 2),
    }

def summarize_korean(
    self,
    text: str,
    max_length: int = 128,
    min_length: int = 20,
    num_beams: int = 4,
) -> dict:
    """ke-t5를 이용한 한국어 문서 요약"""
    # ke-t5는 'summarize: ' prefix 사용
    input_text = f'summarize: {text}'
    inputs = self.ko_tokenizer(
        input_text,
        return_tensors='pt',
        max_length=512,
        truncation=True,
    ).to(self.device)

    with torch.no_grad():
        summary_ids = self.ko_model.generate(
            inputs['input_ids'],
            max_length=max_length,
            min_length=min_length,
            num_beams=num_beams,
            no_repeat_ngram_size=3,
            early_stopping=True,
        )

```

```

summary = self.ko_tokenizer.decode(
    summary_ids[0],
    skip_special_tokens=True,
)
input_tokens = inputs['input_ids'].shape[1]
output_tokens = summary_ids.shape[1]
return {
    'summary': summary,
    'input_tokens': input_tokens,
    'output_tokens': output_tokens,
    'compression_ratio': round(input_tokens / max(output_tokens, 1), 2),
}
}

summarizer_service = SummarizerService()

```

3.2.3 입력 문장 → 요약 결과 반환 API

models/schemas.py - 요약 스키마 추가

```

# models/schemas.py (요약 관련 스키마 추가)

class SummarizeRequest(BaseModel):
    text: str = Field(
        ...,
        min_length=50,
        max_length=5000,
        description='요약할 원문 텍스트 (최소 50자, 최대 5000자)',
    )
    language: Language = Field(
        Language.korean,
        description='원문 언어: ko(한국어) / en(영어)',
    )
    max_length: int = Field(128, ge=30, le=512,
        description='요약문 최대 토큰 길이')
    min_length: int = Field(20, ge=10, le=200,
        description='요약문 최소 토큰 길이')
    num_beams: int = Field(4, ge=1, le=8,
        description='Beam Search 빔 수 (높을수록 고품질, 느림)')

```

```

class Config:
    json_schema_extra = {
        'example': {
            'text': '인공지능(AI)은 인간의 학습, 추론, 지각 등의 인지 능력을 컴퓨터로 구현하는 기술입니다.',
            '최근 딥러닝 기술의 발전으로 자연어 처리, 이미지 인식, 음성 인식 분야에서 ',
            '혁신적인 성과를 거두고 있으며, 의료, 금융, 교육 등 다양한 산업에 활용되고 있습니다.',
            'language': 'ko',
            'max_length': 80,
            'min_length': 20,
            'num_beams': 4
        }
    }

class SummarizeResponse(BaseModel):
    original_text: str
    summary: str
    language: str
    model_used: str
    input_tokens: int
    output_tokens: int
    compression_ratio: float # 압축 비율 (input / output 토큰 수)
    char_count_original: int
    char_count_summary: int

```

[routers/summarizer.py](#)

```

# routers/summarizer.py
from fastapi import APIRouter, HTTPException, status
from models.schemas import SummarizeRequest, SummarizeResponse, Language
from services.summarizer_service import summarizer_service

router = APIRouter(prefix='/summarize', tags=['Summarizer'])

@router.post(
    '/',
    response_model=SummarizeResponse,

```

```

summary='문서 요약',
description='긴 텍스트를 입력하면 핵심 내용만 요약하여 반환합니다.',
)

async def summarize_text(request: SummarizeRequest):
    if request.min_length >= request.max_length:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail='min_length는 max_length보다 작아야 합니다.',
        )

    try:
        if request.language == Language.korean:
            if summarizer_service.ko_model is None:
                raise HTTPException(status_code=503,
                                    detail='한국어 요약 모델 미로드')

            result = summarizer_service.summarize_korean(
                text=request.text,
                max_length=request.max_length,
                min_length=request.min_length,
                num_beams=request.num_beams,
            )

            model_used = 'KETI-AIR/ke-t5-base'
        else:
            if summarizer_service.en_model is None:
                raise HTTPException(status_code=503,
                                    detail='영어 요약 모델 미로드')

            result = summarizer_service.summarize_english(
                text=request.text,
                max_length=request.max_length,
                min_length=request.min_length,
                num_beams=request.num_beams,
            )

            model_used = 'facebook/bart-large-cnn'

    return SummarizeResponse(
        original_text=request.text,
        summary=result['summary'],
        language=request.language.value,
    )

```

```

        model_used=model_used,
        input_tokens=result['input_tokens'],
        output_tokens=result['output_tokens'],
        compression_ratio=result['compression_ratio'],
        char_count_original=len(request.text),
        char_count_summary=len(result['summary']),
    )
except HTTPException:
    raise
except Exception as e:
    raise HTTPException(status_code=500,
                        detail=f'요약 처리 오류: {str(e)}')

@router.get('/health', summary='요약 모델 상태')
async def summarizer_health():
    return {
        'en_model': summarizer_service.en_model is not None,
        'ko_model': summarizer_service.ko_model is not None,
    }

```

3.2.4 길이 제한, max_length 조정, 미리보기 등 기능 포함

요약 품질을 제어하는 가장 중요한 파라미터인 max_length, min_length, num_beams에 대해 더 자세히 살펴봅니다.

파라미터	역할 및 효과	권장 설정값
max_length	생성되는 요약 토큰의 최대 개수. 너무 작으면 내용이 잘림.	원문 길이의 20~30%
min_length	최소 생성 토큰 수. 너무 짧은 요약 방지.	max_length의 30%
num_beams	Beam Search 후보 수. 높을수록 품질 ↑, 속도 ↓	4~6 (균형점)
length_penalty	> 1.0이면 긴 출력 선호. < 1.0이면 짧은 출력 선호.	요약: 1.0~2.0
no_repeat_ngram_size	n-gram 반복 억제. 동일 구절 반복 방지.	2~3
early_stopping	빔 탐색 조기 종료로 속도 향상.	True 권장

다음은 미리보기(Preview) 기능을 추가한 확장 엔드포인트입니다. 원문의 앞부분만 미리보기로 반환하여 사용자가 요약 전 내용을 확인할 수 있게 합니다.

```
# routers/summarizer.py (미리보기 엔드포인트 추가)

@router.post('/preview', summary='요약 전 미리보기',
             description='원문의 앞 200자와 토큰 수를 미리 확인합니다.')
async def preview_text(text: str, language: Language = Language.korean):
    if language == Language.korean:
        tokenizer = summarizer_service.ko_tokenizer
    else:
        tokenizer = summarizer_service.en_tokenizer

    if tokenizer is None:
        raise HTTPException(status_code=503, detail='토큰나이저 미로드')

    token_count = len(tokenizer.encode(text))
    is_truncatable = token_count > 512

    return {
        'preview': text[:200] + ('...' if len(text) > 200 else ''),
        'total_chars': len(text),
        'total_tokens': token_count,
        'will_be_truncated': is_truncatable,
        'max_supported_tokens': 512 if language == Language.korean else 1024,
    }
```

테스트: 한국어 뉴스 요약

```
$ curl -X POST http://localhost:8000/summarize/ \#
-H 'Content-Type: application/json' \#
-d '{
    "text": "정부는 오늘 인공지능 산업 진흥을 위한 대규모 투자 계획을 발표했습니다.
            과학기술정보통신부 장관은 향후 5년간 총 5조 원을 AI 연구개발에 투자하고,
            AI 전문 인력 10만 명을 양성하겠다고 밝혔습니다. 이번 계획에는 의료, 제조,
            금융 분야의 AI 도입 지원과 함께 AI 안전성 확보를 위한 규제 체계 마련도
            포함되어 있습니다."
```

```
"language": "ko",  
"max_length": 60  
'
```

응답 예시:

```
{  
  "original_text": "정부는 오늘 인공지능 산업 진흥을...",  
  "summary": "정부는 5년간 5조 원을 AI 연구개발에 투자하고 전문 인력 10만 명을 양성한다.",  
  "language": "ko",  
  "model_used": "KETI-AIR/ke-t5-base",  
  "input_tokens": 187,  
  "output_tokens": 32,  
  "compression_ratio": 5.84,  
  "char_count_original": 248,  
  "char_count_summary": 45  
}
```

3.3 번역 API 만들기 (MarianMT 기반)

번역(Machine Translation)은 NLP에서 가장 실용적인 태스크 중 하나입니다. 이 절에서는 Helsinki-NLP 그룹이 공개한 MarianMT 모델을 활용하여 다국어 번역 API를 구현합니다.

3.3.1 MarianMT로 영어 ↔ 한국어, 일본어 등 다국어 번역 실습

MarianMT는 Edinburgh 대학교와 Microsoft가 협력하여 개발한 고속 신경망 기계 번역(NMT, Neural Machine Translation) 프레임워크입니다. Helsinki-NLP 그룹은 이를 기반으로 1,000개 이상의 언어 쌍(language pair) 번역 모델을 Hugging Face Hub에 공개했습니다.

언어 쌍	모델 ID	특징
영어 → 한국어	Helsinki-NLP/opus-mt-en-ko	영문 → 한글 번역. 뉴스, 기술 문에 강점.
한국어 → 영어	Helsinki-NLP/opus-mt-ko-en	한글 → 영문 번역.
영어 → 일본어	Helsinki-NLP/opus-mt-en-jap	영문 → 일본어 번역.
영어 → 중국어	Helsinki-NLP/opus-mt-en-zh	영문 → 중국어 간체 번역.
영어 → 프랑스어	Helsinki-NLP/opus-mt-en-fr	영문 → 프랑스어 번역.
다국어 → 영어	Helsinki-NLP/opus-mt-mul-en	100개 이상 언어 → 영문 번역.

MarianMT 아키텍처 특징

- Marian C++ 번역 엔진을 Python/PyTorch로 포팅한 Transformer 기반 Seq2Seq 모델
- 각 언어 쌍마다 별도 모델 파일 (약 300MB ~ 1GB)
- 모델 ID 패턴: Helsinki-NLP/opus-mt-{소스언어}-{목적언어}
- OPUS 병렬 코퍼스로 학습 (위키피디아 번역, OpenSubtitles, CCAligned 등)
- MarianTokenizer 사용 (SentencePiece 기반)
- from_pretrained()으로 tokenizer와 model 모두 로드 가능

3.3.2 모델 예시: Helsinki-NLP/opus-mt-en-ko, opus-mt-ko-en

services/translator_service.py

```

# services/translator_service.py
import torch

from transformers import MarianMTModel, MarianTokenizer
from typing import List, Dict


class TranslatorService:
    """MarianMT 기반 다국어 번역 서비스"""

    # 지원 번역 방향 정의
    SUPPORTED_PAIRS = {
        'en-ko': 'Helsinki-NLP/opus-mt-en-ko',
        'ko-en': 'Helsinki-NLP/opus-mt-ko-en',
        'en-ja': 'Helsinki-NLP/opus-mt-en-jap',
        'en-zh': 'Helsinki-NLP/opus-mt-en-zh',
        'en-fr': 'Helsinki-NLP/opus-mt-en-fr',
        'en-de': 'Helsinki-NLP/opus-mt-en-de',
    }

    def __init__(self):
        self.models: Dict[str, MarianMTModel] = {}
        self.tokenizers: Dict[str, MarianTokenizer] = {}
        self.device = 'cuda' if torch.cuda.is_available() else 'cpu'

    def load_models(self, pairs: List[str] = None):
        """지정 언어 쌍 모델 로드. pairs=None이면 en-ko, ko-en 로드."""
        if pairs is None:
            pairs = ['en-ko', 'ko-en'] # 기본: 한국어-영어 쌍만

        for pair in pairs:
            if pair not in self.SUPPORTED_PAIRS:
                print(f'[WARN] Unsupported pair: {pair}')
                continue

            model_id = self.SUPPORTED_PAIRS[pair]
            print(f'[INFO] Loading translation model: {model_id}')
            self.tokenizers[pair] = MarianTokenizer.from_pretrained(model_id)
            self.models[pair] = MarianMTModel.from_pretrained(

```

```
        model_id
    ).to(self.device)

    self.models[pair].eval()

    print(f'[INFO] Loaded: {pair}')

def translate(
    self,
    text: str,
    source_lang: str,
    target_lang: str,
    num_beams: int = 4,
    max_length: int = 512,
) -> dict:
    pair = f'{source_lang}-{target_lang}'
    if pair not in self.models:
        raise ValueError(
            f'번역 방향 {pair}가 로드되지 않았습니다. '
            f'지원 방향: {list(self.models.keys())}'
        )

    tokenizer = self.tokenizers[pair]
    model = self.models[pair]

    # 입력 토크나이징
    inputs = tokenizer(
        [text],
        return_tensors='pt',
        padding=True,
        truncation=True,
        max_length=512,
    ).to(self.device)

    with torch.no_grad():
        translated_ids = model.generate(
            **inputs,
            num_beams=num_beams,
            max_length=max_length,
```

```

        early_stopping=True,
    )

    translated = tokenizer.batch_decode(
        translated_ids,
        skip_special_tokens=True,
    )

    return {
        'translated': translated[0],
        'source_lang': source_lang,
        'target_lang': target_lang,
        'model_id': self.SUPPORTED_PAIRS[pair],
    }

def translate_batch(
    self,
    texts: List[str],
    source_lang: str,
    target_lang: str,
    batch_size: int = 8,
) -> List[str]:
    """다수 문장 배치 번역"""
    pair = f'{source_lang}-{target_lang}'
    tokenizer = self.tokenizers[pair]
    model = self.models[pair]
    results = []

    for i in range(0, len(texts), batch_size):
        batch = texts[i:i + batch_size]
        inputs = tokenizer(
            batch, return_tensors='pt',
            padding=True, truncation=True
        ).to(self.device)
        with torch.no_grad():
            ids = model.generate(**inputs, num_beams=4)
            decoded = tokenizer.batch_decode(ids, skip_special_tokens=True)
            results.extend(decoded)

```

```

return results

translator_service = TranslatorService()

```

3.3.3 입력 언어 자동 감지 기능 없이 모델별 route 구성

각 번역 방향별로 독립적인 엔드포인트를 제공하거나, 단일 엔드포인트에 `source_lang`, `target_lang` 파라미터를 받는 방식으로 구성합니다. 언어 자동 감지(auto-detect)는 외부 라이브러리(`langdetect` 등)와 추가 모델이 필요하므로 이 절에서는 명시적 언어 지정 방식을 사용합니다.

models/schemas.py - 번역 스키마

```

# models/schemas.py (번역 관련 스키마 추가)

class SupportedLang(str, Enum):
    en = 'en'
    ko = 'ko'
    ja = 'ja'
    zh = 'zh'
    fr = 'fr'
    de = 'de'

class TranslateRequest(BaseModel):
    text: str = Field(
        ...,
        min_length=1,
        max_length=2000,
        description='번역할 원문 텍스트 (최대 2000자)',
    )
    source_lang: SupportedLang = Field(
        SupportedLang.en,
        description='원문 언어 코드 (en/ko/ja/zh/fr/de)',
    )
    target_lang: SupportedLang = Field(
        SupportedLang.ko,

```

description='번역 목적 언어 코드',
)
num_beams: int = Field(4, ge=1, le=8,
description='Beam Search 빔 수')
class Config:
json_schema_extra = {
'example': {
'text': 'Artificial intelligence is transforming every industry.',
'source_lang': 'en',
'target_lang': 'ko',
'num_beams': 4
}
}
class TranslateResponse(BaseModel):
original_text: str
translated_text: str
source_lang: str
target_lang: str
model_used: str
char_count_original: int
char_count_translated: int
class BatchTranslateRequest(BaseModel):
texts: List[str] = Field(..., min_items=1, max_items=50,
description='번역할 텍스트 목록 (최대 50개)')
source_lang: SupportedLang
target_lang: SupportedLang
class BatchTranslateResponse(BaseModel):
original_texts: List[str]
translated_texts: List[str]
source_lang: str
target_lang: str
count: int

routers/translator.py

```
# routers/translator.py
from fastapi import APIRouter, HTTPException, status
from models.schemas import (TranslateRequest, TranslateResponse,
                             BatchTranslateRequest, BatchTranslateResponse)
from services.translator_service import translator_service

router = APIRouter(prefix='/translate', tags=['Translator'])

@router.post(
    '/',
    response_model=TranslateResponse,
    summary='텍스트 번역',
    description='지정 언어로 텍스트를 번역합니다. 지원 방향: en↔ko, en→ja/zh/fr/de',
)
async def translate_text(request: TranslateRequest):
    if request.source_lang == request.target_lang:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail='원문과 목적 언어가 동일합니다.',
        )

    try:
        result = translator_service.translate(
            text=request.text,
            source_lang=request.source_lang.value,
            target_lang=request.target_lang.value,
            num_beams=request.num_beams,
        )
        return TranslateResponse(
            original_text=request.text,
            translated_text=result['translated'],
            source_lang=request.source_lang.value,
            target_lang=request.target_lang.value,
            model_used=result['model_id'],
            char_count_original=len(request.text),
            char_count_translated=len(result['translated']),
        )
```

```

    )
except ValueError as e:
    raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
                        detail=str(e))
except Exception as e:
    raise HTTPException(status_code=500,
                        detail=f'번역 처리 오류: {str(e)}')

```

```

@router.post(
    '/batch',
    response_model=BatchTranslateResponse,
    summary='배치 번역',
    description='여러 텍스트를 한 번에 번역합니다 (최대 50개)',
)

```

```

async def batch_translate(request: BatchTranslateRequest):

```

```

    try:
        translated = translator_service.translate_batch(
            texts=request.texts,
            source_lang=request.source_lang.value,
            target_lang=request.target_lang.value,
        )

```

```

        return BatchTranslateResponse(
            original_texts=request.texts,
            translated_texts=translated,
            source_lang=request.source_lang.value,
            target_lang=request.target_lang.value,
            count=len(translated),
        )

```

```

    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

```

@router.get('/supported-pairs', summary='지원 번역 방향 목록')

```

```

async def get_supported_pairs():

```

```

    loaded = list(translator_service.models.keys())
    all_pairs = translator_service.SUPPORTED_PAIRS
    return {
        'loaded_pairs': loaded,
        'all_supported': list(all_pairs.keys()),
    }

```

```
'models': {k: v for k, v in all_pairs.items() if k in loaded},  
}
```

실습 팁: Swagger UI에서 다국어 입력 테스트

Swagger UI(/docs)에서 번역 API를 테스트할 때 한글, 영어, 일본어 등 다양한 언어를 직접 입력할 수 있습니다.

테스트 예시 입력:

한국어 → 영어: '오늘 날씨가 매우 좋습니다.'

영어 → 한국어: 'The future of AI is bright.'

영어 → 일본어: 'Machine learning is fascinating.'

Swagger UI에서 한글 입력 시 인코딩 오류가 발생하면 curl이나 Postman을 활용하세요.

curl 명령어 예시:

```
curl -X POST http://localhost:8000/translate/ ₩  
-H 'Content-Type: application/json; charset=utf-8' ₩  
-d '{"text": "AI", "source_lang": "en", "target_lang": "ko"}'
```

3.4 문서 유사도 검색 API (FAISS 도입은 간단히)

문서 유사도 검색(Semantic Search)은 단순한 키워드 매칭을 넘어, 텍스트의 '의미(Semantics)'를 벡터로 표현하여 의미적으로 가장 유사한 문서를 찾는 기술입니다. 이 절에서는 sentence-transformers와 Facebook의 FAISS 라이브러리를 조합하여 FAQ 검색, 뉴스 제목 검색 등을 구현합니다.

3.4.1 문장 임베딩 → 벡터 유사도 기반 검색 시스템 구조 설명

의미 검색 시스템의 전체 구조는 크게 '인덱싱 단계'와 '검색 단계'로 나뉩니다.

단계	수행 작업
① 문서 준비	검색 대상 문서(FAQ, 뉴스 제목 등) 목록 준비
② 임베딩 생성	sentence-transformers로 각 문서를 고차원 벡터로 변환
③ FAISS 인덱스 구축	생성된 벡터들을 FAISS에 추가하여 검색 인덱스 구성
④ 쿼리 임베딩	사용자 검색어를 동일 모델로 벡터로 변환
⑤ 유사도 검색	FAISS에서 쿼리 벡터와 가장 유사한 K개 문서 벡터 검색
⑥ 결과 반환	유사도 점수와 함께 가장 관련성 높은 문서 목록 반환

벡터 유사도 측정 방식으로는 코사인 유사도(Cosine Similarity)가 가장 널리 사용됩니다. 두 벡터가 이루는 각도의 코사인 값으로 유사도를 측정하며, 1.0에 가까울수록 의미가 비슷한 문서입니다.

💡 코사인 유사도 vs 유클리드 거리

코사인 유사도 (Cosine Similarity):

- 벡터 방향의 유사성 측정 (크기 무관)
- NLP 임베딩에 가장 적합
- FAISS의 IndexFlatIP (내적) + 정규화로 구현
- 값 범위: -1.0 ~ 1.0 (높을수록 유사)

유클리드 거리 (L2 Distance):

- 두 벡터 간 직선 거리 측정
- FAISS의 IndexFlatL2로 구현
- 값 범위: 0 ~ ∞ (낮을수록 유사)

3.4.2 sentence-transformers 모델로 임베딩 생성

sentence-transformers 라이브러리는 문장 단위의 의미론적 임베딩을 생성하는 데 특화된 모델들을 제공합니다. 기존 BERT 계열 모델을 'Siamese Network' 방식으로 파인튜닝하여 문장 간 유사도 계산에 최적화되었습니다.

모델 ID	특징	임베딩 차원
all-MiniLM-L6-v2	경량(22M). 빠른 속도. 영어 특화. 가장 많이 사용됨.	384
all-mpnet-base-v2	고품질(109M). 영어 최고 성능. GPU 권장.	768
paraphrase-multilingual-MiniLM-L12-v2	50개 언어 지원. 한국어 포함. 범용 다국어.	384
jhgan/ko-sroberta-multitask	한국어 특화. STS, NLI 학습. 한국어 문장 유사도 최고.	768

services/similarity_service.py

```
# services/similarity_service.py
import json
import numpy as np
from typing import List, Dict, Optional
from pathlib import Path

try:
    import faiss
    FAISS_AVAILABLE = True
except ImportError:
    FAISS_AVAILABLE = False
    print('[WARN] FAISS not installed. Using numpy fallback.')
```

```

from sentence_transformers import SentenceTransformer

class SimilarityService:
    """sentence-transformers + FAISS 기반 문서 유사도 검색"""

    MODEL_ID = 'paraphrase-multilingual-MiniLM-L12-v2'

    def __init__(self):
        self.model: Optional[SentenceTransformer] = None
        self.documents: List[Dict] = [] # {'id', 'text', 'metadata'}
        self.embeddings: Optional[np.ndarray] = None
        self.index = None # FAISS 인덱스
        self.embedding_dim: int = 384

    def load_model(self):
        print(f'[INFO] Loading sentence-transformer: {self.MODEL_ID}')
        self.model = SentenceTransformer(self.MODEL_ID)
        self.embedding_dim = self.model.get_sentence_embedding_dimension()
        print(f'[INFO] Embedding dim: {self.embedding_dim}')

    def add_documents(self, docs: List[Dict]):
        """문서 추가 (id, text, metadata 포함)"""
        self.documents.extend(docs)

    def build_index(self, data_path: str = 'data/sample_docs.json'):
        """FAISS 인덱스 구축"""
        # 샘플 데이터 로드
        if not self.documents:
            path = Path(data_path)
            if path.exists():
                with open(path, 'r', encoding='utf-8') as f:
                    self.documents = json.load(f)
            else:
                self.documents = self._get_default_documents()

```

```
texts = [doc['text'] for doc in self.documents]

print(f'[INFO] Building index for {len(texts)} documents...')

# 임베딩 생성
self.embeddings = self.model.encode(
    texts,
    convert_to_numpy=True,
    normalize_embeddings=True, # 코사인 유사도를 위해 정규화
    show_progress_bar=True,
)

if FAISS_AVAILABLE:
    # FAISS IndexFlatIP (내적 = 정규화 후 코사인 유사도)
    self.index = faiss.IndexFlatIP(self.embedding_dim)
    self.index.add(self.embeddings.astype(np.float32))
    print(f'[INFO] FAISS index built. Total: {self.index.ntotal} vectors')
else:
    print('[INFO] Using numpy for similarity (no FAISS)')

def search(
    self,
    query: str,
    top_k: int = 5,
) -> List[Dict]:
    """쿼리와 가장 유사한 문서 검색"""
    if self.model is None:
        raise RuntimeError('모델이 로드되지 않았습니다.')

    # 쿼리 임베딩
    query_embedding = self.model.encode(
        [query],
        convert_to_numpy=True,
        normalize_embeddings=True,
    ).astype(np.float32)

    if FAISS_AVAILABLE and self.index is not None:
        scores, indices = self.index.search(query_embedding, top_k)
```

```

results = []
for score, idx in zip(scores[0], indices[0]):
    if idx < 0:
        continue
    doc = self.documents[idx].copy()
    doc['similarity_score'] = float(score)
    results.append(doc)
else:
    # numpy fallback (코사인 유사도 직접 계산)
    scores = np.dot(self.embeddings, query_embedding.T).flatten()
    top_indices = np.argsort(scores)[::-1][:top_k]
    results = []
    for idx in top_indices:
        doc = self.documents[idx].copy()
        doc['similarity_score'] = float(scores[idx])
        results.append(doc)

return results

def _get_default_documents(self) -> List[Dict]:
    """기본 FAQ 샘플 문서"""
    return [
        {'id': 1, 'text': 'FastAPI는 Python 기반의 고성능 웹 프레임워크입니다.',
         'category': 'FastAPI'},
        {'id': 2, 'text': 'Hugging Face는 AI 모델을 공유하는 플랫폼입니다.',
         'category': 'HuggingFace'},
        {'id': 3, 'text': 'Transformer는 어텐션 메커니즘 기반의 딥러닝 모델입니다.',
         'category': 'AI'},
        {'id': 4, 'text': 'BERT는 양방향 트랜스포머 기반의 언어 모델입니다.',
         'category': 'AI'},
        {'id': 5, 'text': 'GPT는 자동회귀 방식의 텍스트 생성 모델입니다.',
         'category': 'AI'},
        {'id': 6, 'text': 'Docker는 컨테이너 기반의 가상화 플랫폼입니다.',
         'category': 'DevOps'},
        {'id': 7, 'text': 'REST API는 HTTP 기반의 웹 서비스 설계 방식입니다.',
         'category': 'Web'},
        {'id': 8, 'text': 'PyTorch는 딥러닝 모델 학습에 사용되는 프레임워크입니다.',
         'category': 'AI'},
    ]

```

```
{'id': 9, 'text': 'unicorn은 ASGI 기반의 Python 웹 서버입니다.',
  'category': 'FastAPI'},
{'id': 10, 'text': 'sentence-transformers는 문장 임베딩 생성 라이브러리입니다.',
  'category': 'AI'},
]

similarity_service = SimilarityService()
```

3.4.3 FAISS를 이용한 유사도 검색 흐름 (간단 인덱스 구조)

FAISS(Facebook AI Similarity Search)는 Facebook AI Research가 개발한 고성능 벡터 유사도 검색 라이브러리입니다. 수백만 개의 벡터에서도 밀리초 단위로 최근접 이웃(Nearest Neighbor)을 찾을 수 있습니다.

FAISS 인덱스 유형	특징 및 사용 적합 상황
IndexFlatL2	L2 거리 기반 정확한 검색. 소규모(~100K) 데이터에 적합. 메모리 큼.
IndexFlatIP	내적(코사인 유사도) 기반. 정규화된 벡터에서 코사인 유사도와 동일.
IndexIVFFlat	Inverted File 방식. 대규모(~1M) 데이터. 학습 단계 필요. 속도 ↑, 정확도 약간 ↓
IndexHNSW	계층적 그래프 기반. 매우 빠른 검색. 메모리 사용 큼. 프로덕션 권장.
IndexPQ	Product Quantization으로 벡터 압축. 매우 대규모 데이터(~1B). 메모리 ↓

📦 FAISS 설치 방법

CPU 버전:

```
$ pip install faiss-cpu
```

GPU 버전 (CUDA 필요):

```
$ pip install faiss-gpu
```

FAISS가 없을 경우 서비스 코드는 numpy 폴백으로 동작하지만 대규모 데이터에서는 성능이 크게 저하됩니다.

프로덕션 환경에서는 반드시 faiss-cpu 이상을 설치하세요.

3.4.4 문서 목록 중 가장 유사한 문장 반환 API 구현

models/schemas.py - 유사도 검색 스키마

```
# models/schemas.py (유사도 검색 스키마 추가)

class SearchRequest(BaseModel):
    query: str = Field(
        ...,
        min_length=1,
        max_length=500,
        description='검색 쿼리 텍스트',
    )
    top_k: int = Field(5, ge=1, le=20,
        description='반환할 최대 결과 수')
    threshold: float = Field(0.3, ge=0.0, le=1.0,
        description='최소 유사도 점수 (0.0~1.0)')

class Config:
    json_schema_extra = {
        'example': {
            'query': 'FastAPI로 AI 서비스 만들기',
            'top_k': 5,
            'threshold': 0.3
        }
    }

class SearchResult(BaseModel):
    id: int
    text: str
    category: Optional[str] = None
    similarity_score: float

class SearchResponse(BaseModel):
    query: str
    results: List[SearchResult]
```

```

total_found: int
model_used: str
index_size: int

class AddDocRequest(BaseModel):
    documents: List[Dict] = Field(
        ...,
        description='추가할 문서 목록. 각 문서는 id, text 필드 필수.',
    )

```

routers/similarity.py

```

# routers/similarity.py
from fastapi import APIRouter, HTTPException, status
from models.schemas import SearchRequest, SearchResponse, SearchResult
from services.similarity_service import similarity_service

router = APIRouter(prefix='/search', tags=['Semantic Search'])

@router.post(
    '/',
    response_model=SearchResponse,
    summary='의미 기반 문서 검색',
    description='쿼리 텍스트와 의미적으로 가장 유사한 문서를 반환합니다.',
)
async def semantic_search(request: SearchRequest):
    if similarity_service.model is None:
        raise HTTPException(
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
            detail='유사도 검색 모델이 로드되지 않았습니다.',
        )

    try:
        raw_results = similarity_service.search(
            query=request.query,
            top_k=request.top_k,
        )

```

```

# threshold 필터링
filtered = [
    SearchResult(
        id=r['id'],
        text=r['text'],
        category=r.get('category'),
        similarity_score=round(r['similarity_score'], 4),
    )
    for r in raw_results
    if r['similarity_score'] >= request.threshold
]

index_size = (similarity_service.index.ntotal
               if similarity_service.index else
               len(similarity_service.documents))

return SearchResponse(
    query=request.query,
    results=filtered,
    total_found=len(filtered),
    model_used=similarity_service.MODEL_ID,
    index_size=index_size,
)

except Exception as e:
    raise HTTPException(status_code=500,
                        detail=f'검색 오류: {str(e)}')

@router.post('/add-documents', summary='문서 추가 및 인덱스 갱신')
async def add_documents(documents: list):
    """동적으로 문서를 추가하고 인덱스를 재구축합니다."""
    try:
        similarity_service.add_documents(documents)
        similarity_service.build_index()
    return {
        'message': f'{len(documents)}개 문서 추가 완료',

```

```

        'total_documents': len(similarity_service.documents),
    }

except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

@router.get('/stats', summary='인덱스 통계 정보')
async def get_index_stats():
    return {
        'total_documents': len(similarity_service.documents),
        'model_loaded': similarity_service.model is not None,
        'index_built': similarity_service.index is not None,
        'embedding_dim': similarity_service.embedding_dim,
        'faiss_available': FAISS_AVAILABLE,
    }

```

3.4.5 간단한 뉴스 제목 검색, FAQ 검색 등으로 실습

실제 활용 예시로 뉴스 제목 검색 시스템을 구성해봅니다. data/sample_docs.json에 뉴스 제목 데이터를 저장하고 인덱싱합니다.

data/sample_docs.json

```

[
    {"id": 1, "text": "삼성전자, 차세대 AI 반도체 개발 발표", "category": "IT"},
    {"id": 2, "text": "카카오, 생성형 AI 서비스 KoGPT 공개", "category": "IT"},
    {"id": 3, "text": "네이버 클로바, 한국어 LLM 성능 대폭 향상", "category": "IT"},
    {"id": 4, "text": "국내 AI 스타트업 투자 규모 역대 최고치 달성", "category": "경제"},
    {"id": 5, "text": "의료 AI, 암 조기 진단 정확도 95% 돌파", "category": "의료"},
    {"id": 6, "text": "자율주행 AI 기술, 도심 도로 주행 테스트 성공", "category": "자동차"},
    {"id": 7, "text": "교육부, AI 기반 개인화 학습 플랫폼 전국 보급", "category": "교육"},
    {"id": 8, "text": "정부, AI 윤리 가이드라인 및 규제 방안 발표", "category": "정치"},
    {"id": 9, "text": "OpenAI GPT-5 출시 임박, 멀티모달 강화 예정", "category": "IT"},
    {"id": 10, "text": "딥러닝 기반 번역 서비스, 전문 통역사 수준 달성", "category": "IT"}
]

```

실행 및 검색 테스트를 진행합니다.

```
# 뉴스 제목 검색 테스트
```

```
$ curl -X POST http://localhost:8000/search/ ₩
```

```
-H 'Content-Type: application/json' ₩
```

```
-d '{"query": "AI 의료 기술 발전", "top_k": 3, "threshold": 0.3}'
```

```
# 응답 예시:
```

```
{  
  "query": "AI 의료 기술 발전",  
  "results": [  
    {  
      "id": 5,  
      "text": "의료 AI, 암 조기 진단 정확도 95% 돌파",  
      "category": "의료",  
      "similarity_score": 0.7821  
    },  
    {  
      "id": 1,  
      "text": "삼성전자, 차세대 AI 반도체 개발 발표",  
      "category": "IT",  
      "similarity_score": 0.4512  
    },  
    {  
      "id": 8,  
      "text": "정부, AI 윤리 가이드라인 및 규제 방안 발표",  
      "category": "정치",  
      "similarity_score": 0.3891  
    }  
  ],  
  "total_found": 3,  
  "model_used": "paraphrase-multilingual-MiniLM-L12-v2",  
  "index_size": 10  
}
```

3.5 React/Vue 없이 Swagger UI로 먼저 체험하기

프론트엔드 개발 경험이 없더라도, FastAPI가 자동으로 생성해주는 Swagger UI를 통해 모든 API를 브라우저에서 직접 체험할 수 있습니다. 이 절에서는 Swagger UI를 최대한 활용하는 방법과 API 사용자 경험을 향상시키는 다양한 설정 방법을 다룹니다.

3.5.1 Swagger UI 기본 사용법 복습

FastAPI 서버를 실행한 후 브라우저에서 `http://localhost:8000/docs` 로 접속하면 Swagger UI가 열립니다. Swagger UI의 주요 구성 요소와 사용 방법을 정리합니다.

Swagger UI 구성 요소	설명 및 활용 방법
API 제목 및 버전	FastAPI() 생성 시 title, version, description 파라미터로 설정
태그(Tags) 그룹	라우터의 tags 파라미터로 API를 카테고리별 그룹화
엔드포인트 목록	HTTP 메서드(POST/GET 등)와 URL 경로 표시. 클릭하면 확장
Try it out 버튼	클릭하면 입력 필드가 활성화되어 API 직접 실행 가능
Request Body	JSON 형식 입력. example 값이 미리 채워져 있으면 편리함.
Parameters	경로 매개변수, 쿼리 매개변수를 개별 입력란으로 표시
Execute 버튼	API 요청 전송. curl 명령어도 함께 표시해줌.
Response 영역	HTTP 상태 코드, 응답 헤더, 응답 Body를 보기 좋게 출력
Schemas 섹션	Pydantic 모델의 JSON Schema를 하단에 자동 표시

3.5.2 POST, GET 요청 실습 방법

Swagger UI에서 각 HTTP 메서드별 요청을 실행하는 방법을 구체적으로 설명합니다.

GET 요청 실습 방법

1. Swagger UI에서 GET /chat/health 엔드포인트를 클릭합니다.
2. 'Try it out' 버튼을 클릭하여 활성화합니다.
3. 'Execute' 버튼을 클릭합니다.

4. 'Responses' 섹션에서 HTTP 상태 코드와 JSON 응답을 확인합니다.
5. 'Curl' 섹션에 자동 생성된 curl 명령어를 복사해 터미널에서도 테스트해보세요.

POST 요청 실습 방법

6. POST /chat/ 엔드포인트를 클릭합니다.
7. 'Try it out' 버튼을 클릭합니다.
8. 'Request body' 텍스트 영역에 이미 채워진 example 값을 확인합니다.
9. 필요한 경우 값을 수정합니다 (예: message 변경, language 변경 등).
10. 'Execute'를 클릭하고 응답 결과를 확인합니다.

💡 Swagger UI 활용 팁

1. /docs 우측 상단 'Authorize' 버튼: JWT 토큰 등 인증이 필요한 API 테스트 시 사용
2. Schemas 섹션: 하단에 모든 Pydantic 모델의 JSON Schema 자동 문서화
3. 응답 예시 'curl' 복사: 자동 생성된 curl을 복사해 CI/CD 테스트 스크립트로 활용
4. Response 200: 성공 응답 스키마 미리 확인 가능 (response_model 지정 시)
5. Expand All/Collapse All: 스크롤 줄이기 위한 전체 접기/펼치기

3.5.3 입력 예시 사전 등록 (example)으로 UX 향상

Swagger UI에서 API를 처음 접하는 사용자가 어떤 값을 입력해야 하는지 바로 알 수 있도록, 각 Pydantic 모델에 example 값을 미리 등록해두는 것이 중요합니다.

다양한 example 등록 방법

```
from pydantic import BaseModel, Field
from typing import Annotated

# 방법 1: Field의 json_schema_extra 활용 (권장, Pydantic v2)
class ChatRequest(BaseModel):
    message: str = Field(
        ...,
        description='사용자 입력 메시지',
```

```

    json_schema_extra={
        'examples': [
            'What is machine learning?',
            '인공지능에 대해 설명해주세요.',
        ]
    }
)

model_config = {
    'json_schema_extra': {
        'examples': [
            {
                'message': 'Explain deep learning',
                'language': 'en',
                'max_length': 200
            },
            {
                'message': '딥러닝을 쉽게 설명해주세요',
                'language': 'ko',
                'max_length': 150
            }
        ]
    }
}

```

방법 2: 엔드포인트 데코레이터에 openapi_extra 추가

```

@router.post(
    '/',
    openapi_extra={
        'requestBody': {
            'content': {
                'application/json': {
                    'examples': {
                        'English Chat': {
                            'summary': '영어 챗봇 예시',
                            'value': {'message': 'Hello!', 'language': 'en'}
                        },
                        'Korean Chat': {
                            'summary': '한국어 챗봇 예시',

```

```

        'value': {'message': '안녕하세요!', 'language': 'ko'}
    }
}
}
}
}
}
}
}
)
async def chat(request: ChatRequest):
    ...

```

3.5.4 각 API마다 response 모델 지정해서 명확한 출력 제공

response_model을 명시적으로 지정하면 Swagger UI에서 예상 응답 스키마를 미리 확인할 수 있고, FastAPI가 응답 데이터를 자동으로 검증·직렬화합니다. 또한 민감한 필드를 응답에서 자동으로 제외하는 용도로도 활용됩니다.

```

from fastapi import APIRouter
from pydantic import BaseModel
from typing import List, Optional

# ———— 응답 모델 계층 구조 ————

class BaseResponse(BaseModel):
    """모든 API 응답의 기본 클래스"""
    success: bool = True
    message: str = 'OK'

class ChatResponse(BaseResponse):
    user_message: str
    bot_response: str
    language: str
    model_used: str

class ErrorResponse(BaseModel):
    success: bool = False
    error_code: str

```

```

error_message: str
details: Optional[dict] = None

# — response_model과 responses 조합 —
@router.post(
    '/',
    response_model=ChatResponse,
    responses={
        200: {
            'description': '챗봇 응답 성공',
            'model': ChatResponse,
        },
        422: {
            'description': '입력 데이터 유효성 오류',
            'content': {
                'application/json': {
                    'example': {
                        'detail': [{'loc': ['body', 'message'], 'msg': 'value too short'}]
                    }
                }
            },
        },
        503: {
            'description': '모델 로드 실패',
            'model': ErrorResponse,
        },
    },
    summary='챗봇 응답 생성',
    description='**사용법**: message에 텍스트 입력, language로 언어 선택(ko/en).',
)

async def chat(request: ChatRequest) -> ChatResponse:
    ...

```

3.5.5 브라우저에서 쉽게 체험하며 API 개념 숙지

Swagger UI 외에도 FastAPI는 ReDoc이라는 또 다른 문서 뷰어를 제공합니다. 두 인터페이스의 특성을 이해하고 상황에 맞게 활용하세요.

기능	Swagger UI (/docs)	ReDoc (/redoc)
대화형 테스트	Try it out 버튼	읽기 전용
가독성	보통 (접고 펼치기)	매우 높음
요청 예시	표시됨	더 깔끔하게 표시
스키마 문서	하단 Schemas 섹션	오른쪽 패널에 상세
개발 시 사용	API 테스트	레퍼런스 문서 확인
팀 공유	프로토타입 시연	API 명세 공유

Swagger UI와 ReDoc의 기본 경로를 변경하거나 비활성화할 수 있습니다.

```
# main.py - Swagger UI / ReDoc 경로 커스터마이징
app = FastAPI(
    title='NLP Web Services API',
    docs_url='/swagger',      # 기본: /docs
    redoc_url='/api-reference', # 기본: /redoc
    openapi_url='/openapi.json', # 기본: /openapi.json
)

# 프로덕션 환경에서 문서 비활성화 (보안)
import os
app = FastAPI(
    docs_url=None if os.getenv('ENV') == 'production' else '/docs',
    redoc_url=None if os.getenv('ENV') == 'production' else '/redoc',
)
```

마지막으로, 지금까지 구현한 모든 서비스를 통합한 main.py의 최종 버전과 서버 실행 방법을 정리합니다.

```
# 최종 통합 실행
$ uvicorn main:app --host 0.0.0.0 --port 8000 --reload

# 주요 API 엔드포인트 목록:
# POST /chat/          → 챗봇 응답 생성
# POST /summarize/      → 문서 요약
# POST /summarize/preview → 요약 전 미리보기
```

# POST /translate/	→ 텍스트 번역
# POST /translate/batch	→ 배치 번역
# POST /search/	→ 의미 기반 문서 검색
# POST /search/add-documents	→ 문서 추가
#	
# 문서:	
# GET /docs	→ Swagger UI
# GET /redoc	→ ReDoc
# GET /openapi.json	→ OpenAPI 스펙

3.6 3장 프로젝트 requirements.txt 및 실행 가이드

이 장에서 구현한 모든 서비스를 실행하기 위한 전체 패키지 목록과 환경 설정 방법을 정리합니다.

requirements.txt

```
# Web Framework
fastapi==0.111.0
uvicorn[standard]==0.29.0
python-multipart==0.0.9

# AI / NLP Core
torch==2.3.0
transformers==4.41.0
sentence-transformers==2.7.0

# Vector Search
faiss-cpu==1.8.0

# Data
numpy==1.26.4
datasets==2.19.0
huggingface_hub==0.23.0
accelerate==0.30.0

# Utilities
pydantic==2.7.1
python-dotenv==1.0.1
aiofiles==23.2.1
```

.env 파일 설정

```
# .env
HF_TOKEN=hf_XXXXXXXXXXXXXXXXXXXXX # Hugging Face API 토큰 (선택)
HF_HOME=./hf_cache # 모델 캐시 디렉터리
```

```
TRANSFORMERS_OFFLINE=0          # 0: 온라인, 1: 오프라인 캐시만
```

```
ENV=development                 # development / production
```

```
# 모델 로드 설정 (메모리 최적화)
```

```
LOAD_EN_CHAT=true
```

```
LOAD_KO_CHAT=true
```

```
LOAD_SUMMARIZER=true
```

```
LOAD_TRANSLATOR=true
```

```
LOAD_SIMILARITY=true
```

```
# 전체 설치 및 실행
```

```
$ python -m venv venv
```

```
$ source venv/bin/activate
```

```
$ pip install -r requirements.txt
```

```
# 환경변수 로드 후 서버 시작
```

```
$ uvicorn main:app --reload
```

```
# 접속 확인
```

```
$ curl http://localhost:8000/
```

```
# {'status': 'NLP Services Running', 'docs': '/docs'}
```

🎯 3장 학습 정리

FastAPI + DialoGPT/KoGPT2로 영어·한국어 챗봇 API를 구현했습니다.

T5/BART 기반 요약 모델로 문서 요약 API를 만들고 파라미터 조정법을 배웠습니다.

MarianMT로 6가지 언어 쌍 번역 API를 구현하고 배치 번역 기능을 추가했습니다.

sentence-transformers + FAISS로 의미 기반 문서 유사도 검색 시스템을 구축했습니다.

Swagger UI를 통해 프론트엔드 없이 모든 API를 체험하고 문서화하는 방법을 익혔습니다.

Pydantic BaseModel로 요청/응답 스키마를 정의하고 자동 유효성 검사를 적용했습니다.

서비스 레이어 분리 구조로 유지보수 가능한 FastAPI 프로젝트를 완성했습니다.

연습 문제

다음 문제들을 풀어보면서 3장의 핵심 개념을 확인해보세요.

11. DialoGPT와 KoGPT2의 입력 텍스트 포맷 차이를 설명하고, 각 모델에서 EOS 토큰이 왜 필요한지 서술하세요.
12. T5 모델에서 'summarize: ' prefix를 입력 앞에 붙이는 이유는 무엇인가요? T5의 텍스트-투-텍스트 패러다임과 연결하여 설명하세요.
13. MarianMT 모델 ID의 네이밍 규칙을 설명하고, 한국어에서 프랑스어로 번역하는 모델 ID를 추론해보세요.
14. 다음 중 문장 임베딩 유사도 계산에 가장 적합한 FAISS 인덱스 유형은?
 - 14.1. a. IndexFlatL2 (유클리드 거리)
 - 14.2. b. IndexFlatIP (내적, 정규화된 벡터)
 - 14.3. c. IndexPQ (Product Quantization)
 - 14.4. d. IndexIVFFlat (Inverted File)
15. Swagger UI에서 POST 요청을 테스트하는 절차를 5단계로 서술하고, response_model을 지정했을 때의 장점 3가지를 설명하세요.
16. [코딩 실습] 이 장의 번역 API를 확장하여, 입력 텍스트와 함께 glossary(전문용어 사전) 딕셔너리를 받아서 번역 후 특정 단어를 치환해주는 POST /translate/with-glossary 엔드포인트를 구현하세요.

3장 요약

이 장에서는 Hugging Face의 다양한 NLP 모델을 FastAPI와 결합하여 4가지 핵심 웹 서비스를 구현했습니다.

- 챗봇 API: DialoGPT(영어)와 KoGPT2(한국어)를 활용한 무상태 텍스트 생성 챗봇. 생성 전략(Greedy/Beam/Sampling)과 temperature, top-p 파라미터의 역할을 이해했습니다.

- 요약기 API: BART(영어)와 ke-t5(한국어)를 활용한 Seq2Seq 요약 서비스. T5의 prefix 기반 태스크 통합 철학과 max_length, num_beams 파라미터 조정 방법을 학습했습니다.
- 번역 API: MarianMT 기반 다국어 번역 서비스. 단일/배치 번역 엔드포인트 구성과 언어 쌍별 모델 관리 방법을 익혔습니다.
- 유사도 검색 API: sentence-transformers로 문장 임베딩을 생성하고 FAISS로 고속 유사도 검색을 구현. 코사인 유사도 기반 의미 검색의 원리를 이해했습니다.
- Swagger UI 활용: response_model, example, tags, description을 활용하여 사용하기 쉬운 API 문서를 자동 생성하는 방법을 학습했습니다.

□ 다음 장 예고: 제4장 이미지 처리 웹 서비스 만들기

4장에서는 Hugging Face의 컴퓨터 비전(CV) 모델을 FastAPI와 연동합니다.

주요 내용:

- 이미지 업로드를 처리하는 FastAPI multipart/form-data 처리
- ViT(Vision Transformer)를 이용한 이미지 분류 API
- DETR을 이용한 객체 탐지(Object Detection) API
- BLIP-2를 이용한 이미지 캡셔닝 API
- Stable Diffusion을 이용한 텍스트-투-이미지 생성 API
- 이미지 파일 저장과 Base64 인코딩 응답 처리